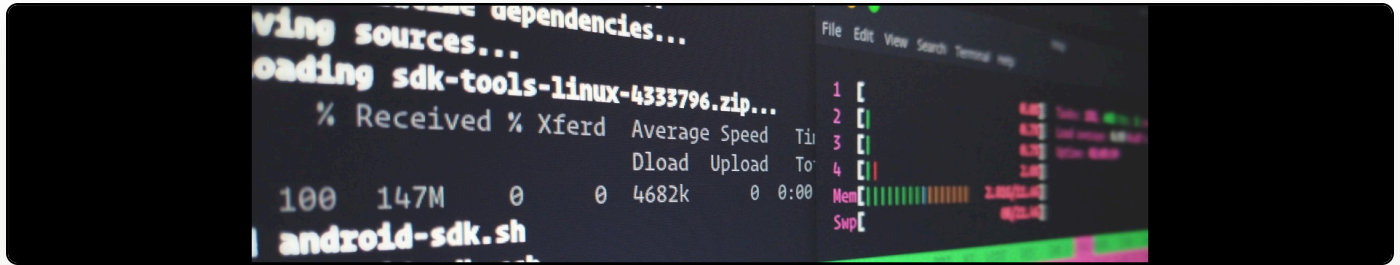


Object Thinking, Systems Description Languages and the Future of the Book

Cite as: Martin Paul Eve, "Object Thinking, Systems Description Languages and the Future of the Book", *eve.gd: Martin Paul Eve*, March 02, 2015, <https://doi.org/10.59348/dvj7h-04x29>.



I am reading a most remarkable book.

I tend to read in sets of three: a novel, a work of lit crit/philosophy/history, then something about technology. This latter category includes library information science stuff, history of technology stuff and sometimes programming manuals. David West's *Object Thinking*, published by Microsoft, is ostensibly in this last group but like nothing else. It traces a history of programming as "systems description languages", that is, as descriptive decomposition tools for real-world problem spaces. He cites a postmodern, by which he means hermeneutic in opposition to formalist, turn in computational thought and gives an introductory bibliography to Husserl. Sure, the book annoyingly has to keep justifying itself to a presumed utilitarian reader for its historical and philosophical approach. It's also a little kitsch in some of its headings. It is still extremely interesting though.

West's fundamental contention is that better programmers, at least in high-level application spaces, need to *think better* (in his case, in a more object-orientated way) in order to write better code. It's not just about adopting specific languages (although there may be interplay between a language and the type of thinking that is encouraged), learning specific tools or any other type of purely

pragmatic change. It's about philosophy. When people struggle with a new programming language, for instance, West contends that this is a disjunct between a person's thought and his or her ability to adequately express it in the language.

West also touches, at one point, though upon the book as an object ripe for modelling in an object-thinking paradigm. "What", West asks on pages 102-103, "are the talents or skills possessed by a book?" "What services can or does it provide?" West then goes on to define a book as:

- A collection of page objects
 - Working with those page objects, it maintains order (sequence).
- An object capable of identifying itself
- An object that can describe itself
 - The description is a separate object that can provide individual elements of the description, such as date published, author or authors, publisher, and so on.
- An object that can provide the reader with access to a specific page or a group of pages (chapters) upon request (also in collaboration with the page or chapter objects).
- An object that acts as a front for a community of objects (pages, tables of contents, indexes, chapters, and the like), allowing users of the community a convenient point of contact. Requests can be sent to the book, knowing that the book will relay those requests to the actual object capable of responding, without interference.
- At one point in its life (in the days when it was a mere unpublished manuscript), an object that was able to add pages to and delete pages from its collection and still maintain the proper order.

The above is all direct citation of West and it is a fascinating way of decomposing the book for implementation within an object-orientated framework.

And yet, systems description languages that model real-world objects are inherently conservative unless they are applied to imaginary spaces. What is the point of remodelling the world inside a simulated (computer) environment, unless we imagine what the new technological capacity can do to improve upon the limitations that were inherent within the object space? Why should our book

be a “collection of page objects” instead of “a collection of paragraphs or sentences or words or characters”? Why should our book maintain sequence? Perhaps there is advantage in allowing readers to reconfigure the sequence of reading, something that wasn’t possible in the print space. Could not a book be defined as an object of communication, both author to reader and peer-to-peer in annotation space? Again, not practical in the object world but very much so in the virtual environment.

This type of thinking consistently hinders our expectations of what a new future book might be. We model our expectations of how we might use a new digital book object on a real-world correlative, even if there might be superior aspects in its digital evolution. At the same time, its opposite (in which techno-fetishism reigns), sees an environment where technological optimists proclaim the future based on what they can build and then find themselves disappointed when their technology is not adopted (because it doesn’t fit with the social paradigm and expectations). In other words, although it is helpful for implementation and decomposability, object thinking, like ordinary language philosophy, has a conservative bias that may be unhelpful in thinking through the technological future of the book.